PostgreSQL

Complete PostgreSQL Full Course

Instructor:
Abu Noman Basar
Software Engineer (Python), eAppair Ltd
Django Instructor at aiQuest Intelligence

Industry Impact of PostgreSQL

Spotify

Manages user data, playlists, and analytics.

Reddit

Supports the core database needs of the social platform.

Netflix

Integrates PostgreSQL into its technology stack.

Uber

Utilizes PostgreSQL in its infrastructure.

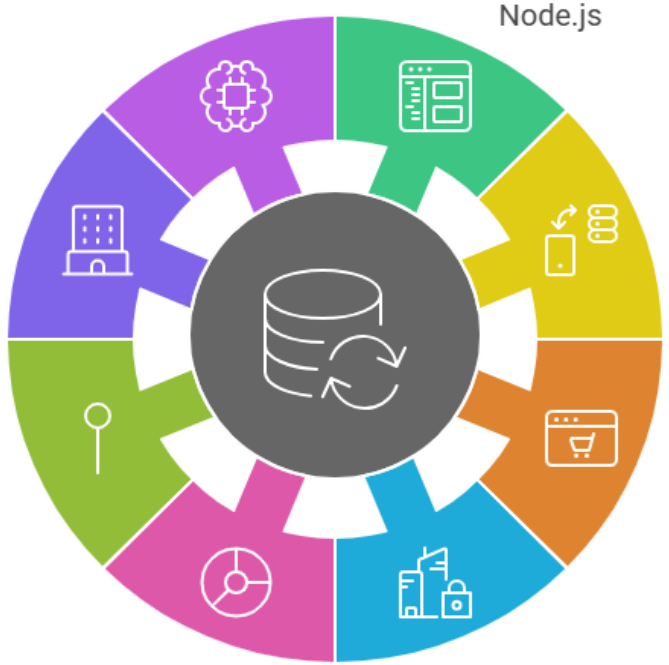
Skype

Employs PostgreSQL for backend operations.

GitLab

Chooses PostgreSQL for its reliability and flexibility.

Where Use PostgreSQL?



Machine Learning
Stores data for ML models

Web Applications
Used with frameworks like Django and Node.js

Mobile App Backends
Stores data for food delivery and social media apps

E-commerce Platforms
Manages orders, inventory, and payments

Banking and Finance
Ensures reliable transaction management

Data Analysis
Supports large datasets and analytical queries

GIS
Enables maps and GPS tracking

Enterprise Applications
Used in ERP and CRM systems

What is PostgreSQL?

- A database is an organized system that stores, manages, and retrieves data efficiently.
- PostgreSQL is a powerful, open-source relational database management system (RDBMS) known for reliability, scalability, and advanced features.

Relational

Data stored in tables

Ex:

Non-Relational(NoSQL)

Data not stored in tables

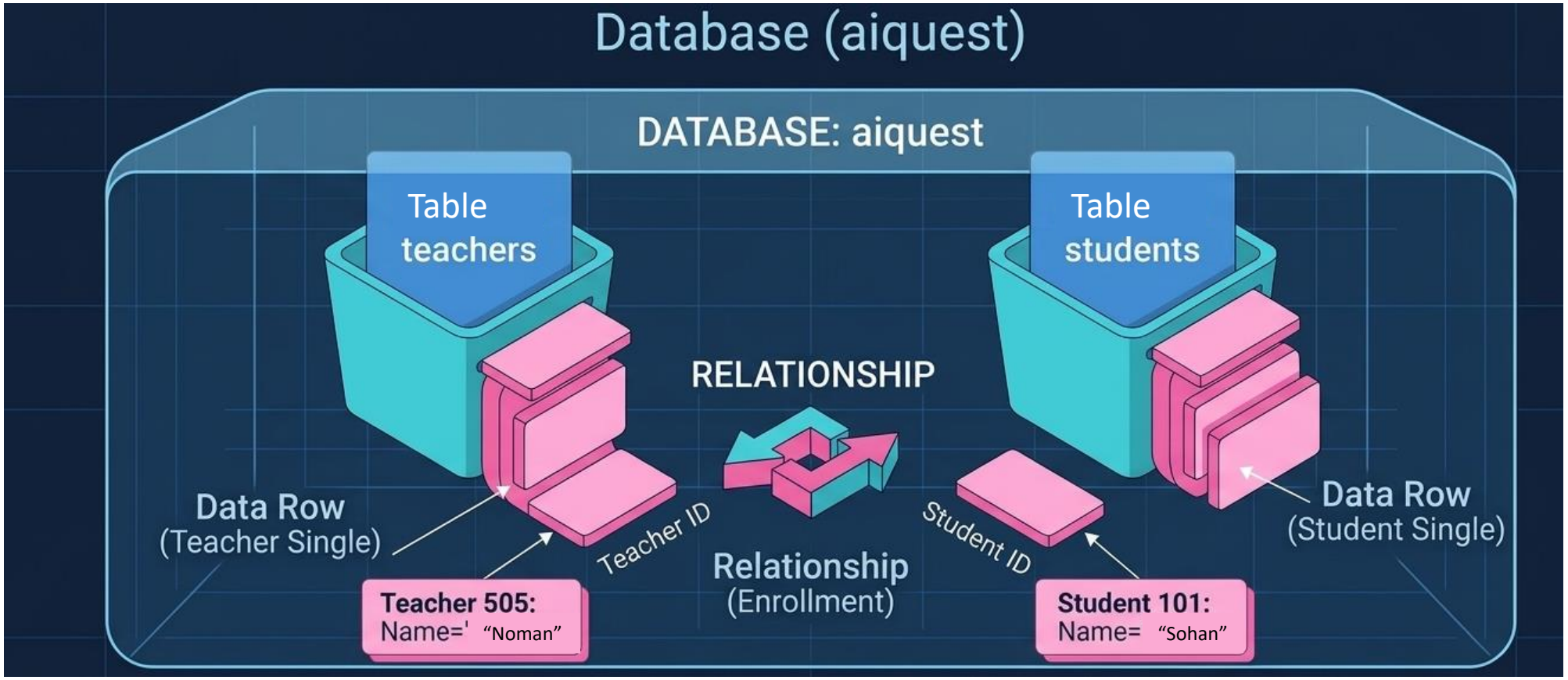
Ex:

Roadmap of PostgreSQL

- ❑ PostgreSQL Setup
- ❑ Filtering & Sorting
- ❑ Aggregation & Grouping
- ❑ Joins & Relationships
- ❑ Data Modification Commands
- ❑ Advanced SQL Topics
- ❑ Cloud Server

Course module: https://github.com/aiQuest-Intelligence/Complete-PostgreSQL

Database


Database (aiquest)
DATABASE: aiquest
Table teachers
Table students
RELATIONSHIP
Data Row (Teacher Single)
Teacher ID
Relationship (Enrollment)
Student ID
Data Row (Student Single)
Teacher 505: Name=' "Noman"
Student 101: Name= "Sohan"

Table

Teacher table

Id	Teacher Name	Course Name	Duration	Seat
101	Noman	Django	4 Month	20
102	Zarin	Data Analysis	3 Month	20
103	Azizul	Python	3 Month	20
104	Kawchar	AI	3 Month	20

Update Data

```
UPDATE teacher
        SET course_name = ‘python programming Language’
WHERE
        id = 102;
```

```
UPDATE teacher
        SET seat = 25
WHERE
        teacher_name = ‘Noman’
```

Delete Data

```
DELETE FROM teacher

WHERE
        id = 102;
```

Data Types

PostgreSQL data types are predefined formats that specify the kind of data a column in a database table can store.

Numeric Data Types

Data Type	Storage	Range	Use Case
SMALLINT	2 bytes	-32,768 to 32,767	Small numbers, flags
INTEGER	4 bytes	-2.1B to 2.1B	General IDs, age, count
BIGINT	8 bytes	±9.2 quintillion	Large IDs, timestamps
DECIMAL/NUMERIC	Variable	131,072 digits	Money, precise calculations
REAL	4 bytes	6 decimal digits	Scientific data
DOUBLE PRECISION	8 bytes	15 decimal digits	High precision math

Character / Text Data Types

CHAR(n)

Fixed-length string. Always stores exactly n characters — pads the rest with spaces. Writing ' aiquest ' in CHAR(10) stores ‘aiquest '.

Country code, Gender (M/F), Fixed format code

VARCHAR(n)

Variable-length string, up to n characters. Uses only as much space as needed. The most commonly used text type in practice.

Name, Email, Address, Title

TEXT

Unlimited length string. No size limit — ideal for large paragraphs, articles, or long descriptions.

Blog post, Comment, Description

Date & Time

DATE

`'2024-01-15'`

Date only. No time component.

Date of birth, Event date

TIME

`'14:30:00'`

Time only. No date component.

Store opening hours, Class schedule

TIMESTAMP

`'2024-01-15 14:30:00'`

Date + Time together.

Order created_at, Log entry

TIMESTAMPTZ

`'2024-01-15 14:30:00+06'`

Timestamp with time zone. Best practice for global apps.

Global app, Meeting time

INTERVAL

`'2 years 3 months'`

A duration or difference between two times.

Subscription period, Age calculation

Boolean & Special

BOOLEAN

`TRUE / FALSE / NULL`

True or false value. Accepts 'yes', 'no', 'on', 'off', '1', '0' — PostgreSQL handles all aliases.

📌 *is_active, is_verified, in_stock*

UUID

`'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11'`

Universally Unique Identifier — a 128-bit random and globally unique ID.

📌 *Primary key, Session ID, API Key*

JSON / JSONB

`'{"name":"Alice","age":25}'`

Stores JSON data. JSONB stores it in binary format — enabling fast search and indexing.

📌 *Flexible schema, Config, Metadata*

ARRAY

`ARRAY['a','b','c'] or '{1,2,3}'::int[]`

Multiple values in a single column. Arrays of any data type are supported.

📌 *Tags, Skills list, Multiple values*

SERIAL — Auto-Increment Types

What is SERIAL?

SERIAL is not a true data type — it is a shorthand that auto-creates a SEQUENCE and sets the default value. It automatically generates a unique integer for each new row inserted.

SMALLSERIAL

2 bytes

1 to 32,767

Small lookup tables

SERIAL

4 bytes

1 to 2,147,483,647

General purpose IDs

BIGSERIAL

8 bytes

1 to 9.2 quintillion

Large-scale systems

Usage Example:

```
CREATE TABLE products (
  id      SERIAL       PRIMARY KEY,  -- auto: 1, 2, 3 ...
  name    VARCHAR(100) NOT NULL
);
```

Real-World Example — E-commerce Users Table

```sql
CREATE TABLE users (
  id          UUID         DEFAULT gen_random_uuid() PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(255) UNIQUE NOT NULL,
  age         SMALLINT,
  balance     DECIMAL(10,2) DEFAULT 0.00,
  is_active   BOOLEAN      DEFAULT TRUE,
  tags        TEXT[],              -- Array type
  preferences JSONB,               -- JSON Binary
  created_at  TIMESTAMPTZ  DEFAULT NOW()
);
```

When to Use Which Type?

Money or Financial data?

→ DECIMAL/NUMERIC — never use FLOAT!

Choosing a Primary Key?

→ UUID (distributed) or SERIAL/BIGSERIAL (simple)

Storing Date or Time?

→ TIMESTAMPTZ — always include timezone, best practice

Is the text length fixed?

→ VARCHAR(n) — if unknown length, use TEXT

Yes / No type of data?

→ BOOLEAN — don't use '1'/'0' strings

Flexible or Dynamic data?

→ JSONB — faster and indexable vs plain JSON

Summary

Numeric:	SMALLINT, INTEGER, BIGINT, DECIMAL, REAL, DOUBLE PRECISION
Text:	CHAR(n) → fixed, VARCHAR(n) → variable, TEXT → unlimited
Date/Time:	DATE, TIME, TIMESTAMP, TIMESTAMPTZ, INTERVAL
Special:	BOOLEAN, UUID, JSON/JSONB, ARRAY
Auto-ID:	SMALLSERIAL, SERIAL, BIGSERIAL — auto-increment integers

✅ **Choosing the right data type means — better performance, less storage, fewer bugs!**

What is a Primary Key?

Definition

A Primary Key is a column that uniquely identifies every row in a table. No two rows can have the same primary key value, and it can never be NULL.

Real-Life Analogies

National ID Card

Every citizen has a unique NID number — no two people share one.

Phone Number

Every phone number is unique — you can find exactly one person.

Order Number

Every e-commerce order has a unique order ID — no duplicates.

Rules & Properties of Primary Key

🔒 UNIQUE

Every value in the primary key column must be unique. No two rows can share the same PK value.

```
id = 1, id = 2, id = 3 ✓     id = 1, id = 1 ✗
```

⊘ NOT NULL

A primary key column can never be NULL. Every row MUST have a PK value — no exceptions.

```
id = 5 ✓     id = NULL ✗
```

1 ONE PER TABLE

Each table can have only ONE primary key. It may span multiple columns (composite), but still counts as one PK.

```
PRIMARY KEY (id) ✓     Two separate PKs ✗
```

⚡ AUTO INDEXED

PostgreSQL automatically creates a unique index on the primary key column — making lookups extremely fast.

```
SELECT * FROM users WHERE id = 5  → instant ⚡
```

SERIAL / INTEGER as Primary Key

How it works

1. SERIAL is shorthand — PostgreSQL internally creates a SEQUENCE object.
2. Each INSERT auto-gets the next number: 1, 2, 3, 4 ...
3. SERIAL = INTEGER + auto-increment (4 bytes, up to ~2.1 billion).
4. BIGSERIAL for very large tables (8 bytes, up to ~9.2 quintillion).
5. Best for: small-to-medium apps, simple projects, internal IDs.

Using SERIAL:

```
CREATE TABLE users (
  id    SERIAL       PRIMARY KEY,
  name  VARCHAR(100) NOT NULL
);
```

Modern approach (PostgreSQL 10+):

```
CREATE TABLE users (
  id   INT GENERATED ALWAYS AS
             IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL
);
```

UUID as Primary Key

Example UUID: `'a3f1c2d4-9b8e-4f3a-b2c1-000000000001'`

✓ Why use UUID?

- ✓ Globally unique — safe across multiple databases & servers
- ✓ IDs can be generated in the app before hitting the database
- ✓ Harder to guess — better security for public-facing APIs
- ✓ No ID collision when merging data from different sources

⚠ Downsides to know

- → Larger storage — 16 bytes vs 4 bytes for INTEGER
- → Slightly slower index performance (random insertion)
- → Harder to read/debug in logs: 'a3f1...' vs just '42'
- → Use UUIDv7 or ULID for better performance (time-sorted)

UUID — Code Examples

Enable the uuid-ossp extension first:

```
-- Enable extension once per database
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

CREATE TABLE orders (
  id         UUID  DEFAULT uuid_generate_v4()  PRIMARY KEY,
  product    VARCHAR(200) NOT NULL
);
```

Using built-in gen_random_uuid() — no extension needed (PostgreSQL 13+):

```
CREATE TABLE orders (
  id         UUID  DEFAULT gen_random_uuid()  PRIMARY KEY,
  product    VARCHAR(200) NOT NULL
);
```

Tip: For new projects use gen_random_uuid() — it's built-in and requires no extra extension.

Composite Primary Key

A Composite Primary Key uses TWO or more columns together to uniquely identify a row. Neither column alone is unique — but the COMBINATION is.

Example: student_courses table

student_id	course_id	grade
101	CS101	A
101	CS201	B+
102	CS101	A-
102	CS201	B

← These two columns together form the PRIMARY KEY

SQL:

```sql
CREATE TABLE
  student_courses (

  student_id INT,
  course_id  VARCHAR(20),
  grade      CHAR(2),

  PRIMARY KEY
    (student_id,
     course_id)
);
```

PRIMARY KEY vs UNIQUE vs NOT NULL

Feature	PRIMARY KEY	UNIQUE	NOT NULL
Uniqueness	✅ Required	✅ Required	❌ Not enforced
NULL allowed?	❌ Never NULL	✅ One NULL ok	❌ Never NULL
How many per table?	1 only	Many allowed	Many allowed
Auto index created?	✅ Yes	✅ Yes	❌ No
Can span columns?	✅ Yes (composite)	✅ Yes	✅ Yes
Typical use	Row identifier	Email, username	Required fields

Common Mistakes to Avoid

✗ Using VARCHAR or TEXT as PK

Why: String PKs are slow to index and compare. Use integers or UUIDs.

✓ Fix: Use SERIAL, INT, or UUID instead.

✗ Using email or username as PK

Why: Emails/usernames can change. A PK should never change once set.

✓ Fix: Use a surrogate key (SERIAL or UUID) as PK. Add UNIQUE to email.

✗ No Primary Key on a table

Why: Without a PK, rows can't be uniquely identified. Updates and deletes become dangerous.

✓ Fix: Always define a PRIMARY KEY on every table, no exceptions.

✗ Using non-meaningful composite PKs

Why: Combining 3+ columns as PK makes foreign keys complex and joins slow.

✓ Fix: Add a surrogate SERIAL/UUID PK and mark the combo UNIQUE instead.

Best Practices & Real-World Code

Always add a PK

Every table, no exception

SERIAL for simple apps

Fast, easy, no config needed

UUID for distributed

Multi-server / microservices

Never use email as PK

Data can change — PKs don't

Production-ready example — e-commerce orders table:

```
CREATE TABLE orders (
  id            UUID          DEFAULT gen_random_uuid()  PRIMARY KEY,
  order_number  SERIAL        UNIQUE,                          -- human-readable
  user_id       UUID          NOT NULL,                       -- FK to users
  total_amount  DECIMAL(12,2)  NOT NULL  DEFAULT 0.00,
  status        VARCHAR(20)   DEFAULT 'pending',
  created_at    TIMESTAMPTZ   DEFAULT NOW()
);
```

Summary

Primary Key

Uniquely identifies every row. Must be UNIQUE and NOT NULL. One PK per table.

Rules

Cannot be NULL, cannot duplicate, auto-indexed by PostgreSQL for speed.

SERIAL

Best for simple projects. Auto-increments: 1, 2, 3 ... Easy and fast.

UUID

Best for distributed systems. Globally unique. Use gen_random_uuid().

Composite PK

Two or more columns together. Use for junction/mapping tables.

Common Mistakes

Never use email as PK. Never skip a PK. Don't use VARCHAR as PK.

A well-chosen Primary Key = faster queries, cleaner data, and a solid database foundation.

Complete CRUD Reference — aiquest_teacher Table

CREATE

```
CREATE TABLE teachers (
  id SERIAL PRIMARY KEY, ...
);
```

INSERT

```
INSERT INTO teachers
  (teacher_name, course_name,
   duration_months, seat)
VALUES ('Noman', ...);
```

SELECT

```
SELECT * FROM teachers;
```

UPDATE

```
UPDATE teachers
SET duration_months = 5
WHERE id = 1;
```

DELETE

```
DELETE FROM teachers
WHERE id = 3;
```